

DotNetNuke MVC Modul Fejlesztése C#-ban

Campus Events – egy konkrét modulfejlesztési példa

Készítette: Boglári Gergely

Tartalom

1. Bevezetés – Mi az a DotNetNuke?
 2. MVC minta
 3. A fejlesztői környezet felkészítése
 4. Projektstruktúra Visual Studióban
 5. Adatbázis tervezés és SQL scriptek
 6. A Model réteg
 7. A DataProvider réteg
 8. A Service réteg
 9. A Controller
 10. A View-k
 11. A Manifest fájl (.dnn)
 12. Build és Deploy
 13. Tipikus hibák és tanulságok
 14. API használat DNN MVC modulban
 15. Fájlfeltöltés és fájlkezelés DNN-ben
-

1. Bevezetés – Mi az a DotNetNuke?

A DotNetNuke, vagy röviden DNN, egy nyílt forráskódú tartalomkezelő rendszer (CMS), amelyet a .NET platformon fejlesztettek ki, és amelyet elsősorban vállalati, intézményi weboldalak üzemeltetésére használnak.

A DNN lényegében egy keretrendszer, amely lehetővé teszi, hogy a weboldal tartalmát, kinézetét és funkcionalitását **modulok** segítségével bővítsük. Minden funkció, legyen az hírlevél, eseménynaptár, fórum vagy bármi más, egy önálló, cserélhető darabként létezik az oldalon.

Miért fontos ez nekünk?

Amikor DNN-be fejlesztünk, nem egy hagyományos webalkalmazást írunk. Nem mi hozzuk létre az egész oldalt, hanem egyetlen, jól meghatárolt funkcióegységet, **modult** illesztünk bele egy már meglévő rendszerbe.

A DNN hosszú éveken át WebForms-alapú modulokat (ASCX) támogatott, és sok régi rendszer ma is így működik. Aztán a DNN 9.x-es verziójával megjelent az igazi MVC támogatás, ami lehetővé tette, hogy modern, tesztelhető, jól strukturált kódot írjunk.

2. MVC minta

Az MVC (Model-View-Controller) egy architektúráis tervezési minta, amely a felelősségeket három elkülönülő rétegre osztja:

- **Model** – az adatok és az üzleti logika reprezentációja
- **View** – a megjelenítési réteg, amit a felhasználó lát
- **Controller** – a logika, amely a kéréseket fogadja és koordinálja

Ha valaki már fejlesztett ASP.NET MVC alkalmazást, az MVC működése ismerős lesz. Ami viszont DNN-specifikus, az a módszer, ahogyan ez az egész befészkei magát a CMS rendszerbe.

Hogyan fut le egy kérés DNN-ben?

Normál ASP.NET MVC esetén a folyamat egyszerű: HTTP-kérés jön, a routing meghatározza a controllert és az actiont, az action visszaad egy View-t, és kész. DNN-ben ez kicsit összetettebb, mert a DNN saját routing és modul-felismerő mechanizmusa is közbeszól.

Amikor egy DNN oldalon betöltődik egy MVC modul, a DNN framework megkeresi a megfelelő controllert az URL-ben lévő mid (Module ID) és a route adatok alapján. Egy tipikus DNN MVC URL valahogy így néz ki:

`https://pelda-oldal.hu/TabId/42/ctl/MVC/mid/128/controller/Event/action/Index`

Ez elsőre meglepő lehet, de lényegében ugyanaz történik, mint egy normál MVC alkalmazásnál – csak a DNN hozzáadja a saját kontextusát (melyik oldalon, melyik modulban vagyunk).

A DNN MVC Controller különlegessége

A leglényegesebb különbség az, hogy a DNN MVC controllerünk nem `System.Web.Mvc.Controller`-ből öröklődik, hanem a DNN saját `DnnController`-éből:

```
public class EventController : DnnController
{
    // ...
}
```

A `DnnController` maga is `Controller`-ből öröklődik, de rárakja a DNN-specifikus funkciókat, hozzáférést biztosít a portál kontextushoz, a modul beállításaihoz, a felhasználói jogosultságokhoz.

3. A fejlesztői környezet felkészítése

Mielőtt egyetlen sor kódot írnánk, be kell állítanunk a fejlesztői környezetet. Ez általában a következőkből áll:

Szükséges szoftverek

- **Visual Studio 2022** (Community kiadás is megfelel)
- **SQL Server** (LocalDB is elegendő fejlesztéshez)
- **DNN Platform** – lokálisan telepítve (jellemzően IIS-en futtatva)
- **DNN Module Development Template** – egy Visual Studio projekt sablon

DNN lokális telepítése

A DNN lokális telepítése önmagában egy hosszabb folyamat, amelyet most nem részletezünk. A lényeg: a fejlesztéshez szükségünk van egy futó DNN példányra, amelybe a modulunkat majd be tudjuk tölteni.

Visual Studio projekt sablon

A DNN Foundation biztosít egy projekt sablonkészletet (DNN Module Templates), amelyek letölthetők a Visual Studio Marketplace-ről. Ezek a sablonok automatikusan beállítják a fájlstruktúrát, az assembly hivatkozásokat és az alapvető konfigurációt.

A mi esetünkben **DNN MVC Module** sablont választjuk.

4. Projektstruktúra Visual Studióban

Amikor létrehozunk egy DNN MVC modult, a projektünk egy jól meghatározott mappahierarchiát kap. Nézzük meg ezt a mi CampusEvents modulunk példáján keresztül.

```
CampusEvents/
├── Controllers/
│   └── EventController.cs
├── Models/
│   └── EventInfo.cs
├── Views/
│   └── Event/
│       ├── Index.cshtml
│       ├── Details.cshtml
│       ├── Edit.cshtml
│       └── Delete.cshtml
└── Services/
    └── EventService.cs
```



Miért így van felépítve?

A DNN keresni fog bizonyos fájlokat bizonyos helyeken:

- A .dnn manifest fájlnek a gyökérben kell lennie
- Az SQL scriptek elérési útjára a manifest hivatkozik
- A View-knak a Views/{ControllerNév}/ mappában kell lenniük, pontosan úgy, mint normál ASP.NET MVC-ben

5. Adatbázis tervezés és SQL scriptek

Minden komolyabb modul saját adatbázis táblákkal dolgozik. A DNN-ben az a bevett gyakorlat, hogy az adatbázis-táblák neve egy prefix-szel kezdődik (alapértelmezés szerint dbo.), és a DNN saját táblanevektől való megkülönböztetésük érdekében szokás modulspecifikus prefixet is használni.

A mi CampusEvents modulunk eseményeket kezel, egy eseménynek van neve, leírása, helyszíne, dátuma és egy szervező felhasználó-azonosítója.

Az SQL DataProvider fájl

A DNN az SQL scripteket .SqlDataProvider kiterjesztésű fájlokban várja. A fájl neve megfelel a modul verziószámának (ez a verziókövetéshez is fontos):

Fájl: 01.00.00.SqlDataProvider

```

/*****
/*****                               SqlDataProvider                               *****/
/*****                               *****/

```

```

/***** Joomla-inspired DNN Module: CampusEvents *****/
/***** Verzió: 01.00.00 *****/
/*****

IF NOT EXISTS (SELECT * FROM sysobjects WHERE id =
OBJECT_ID(N'{databaseOwner}[{objectQualifier}CampusEvents]'))
BEGIN
    CREATE TABLE {databaseOwner}[{objectQualifier}CampusEvents] (
        [EventId] INT IDENTITY(1,1) NOT NULL,
        [ModuleId] INT NOT NULL,
        [Title] NVARCHAR(200) NOT NULL,
        [Description] NVARCHAR(MAX) NULL,
        [Location] NVARCHAR(300) NULL,
        [EventDate] DATETIME NOT NULL,
        [CreatedByUserId] INT NOT NULL,
        [CreatedOnDate] DATETIME NOT NULL DEFAULT GETDATE(),
        [LastModifiedByUserId] INT NULL,
        [LastModifiedOnDate] DATETIME NULL,
        CONSTRAINT [PK_CampusEvents] PRIMARY KEY CLUSTERED ([EventId] ASC)
    )
END
GO

-- Index a ModuleId-ra, mert mindig szűrünk rá
IF NOT EXISTS (SELECT * FROM sysindexes WHERE id =
OBJECT_ID(N'{databaseOwner}[{objectQualifier}CampusEvents]') AND name =
N'IX_CampusEvents_ModuleId')
BEGIN
    CREATE INDEX [IX_CampusEvents_ModuleId]
    ON {databaseOwner}[{objectQualifier}CampusEvents] ([ModuleId])
END
GO

```

A {databaseOwner} és {objectQualifier} tokeneket a DNN automatikusan helyettesíti a telepítés során. Ez teszi lehetővé, hogy a modul különböző DNN telepítéseken is működjön, ahol esetleg más a sémanév vagy más prefix-et használnak.

Stored Procedures

A DNN DataProvider minta hagyományosan tárolt eljárásokat (stored procedure-öket) használ az adatbázis-műveletekhez. Ez több okból is jó döntés, hiszen a lekérdezések újrafelhasználhatók, könnyebben tesztelhetők, és az adatbázis-logika nem keveredik a C# kóddal.

SELECT – Események listázása

```

IF EXISTS (SELECT * FROM sysobjects WHERE id =
OBJECT_ID(N'{databaseOwner}[{objectQualifier}CampusEvents_GetEvents]'))
    DROP PROCEDURE {databaseOwner}[{objectQualifier}CampusEvents_GetEvents]
GO

```

```

CREATE PROCEDURE {databaseOwner}[{objectQualifier}CampusEvents_GetEvents]
    @ModuleId INT
AS
BEGIN
    SET NOCOUNT ON;

    SELECT
        EventId,
        ModuleId,
        Title,
        Description,
        Location,
        EventDate,
        CreatedByUserId,
        CreatedOnDate,
        LastModifiedByUserId,
        LastModifiedOnDate
    FROM {databaseOwner}[{objectQualifier}CampusEvents]
    WHERE ModuleId = @ModuleId
    ORDER BY EventDate ASC
END
GO

```

SELECT – Egyetlen esemény lekérdezése

```

IF EXISTS (SELECT * FROM sysobjects WHERE id =
OBJECT_ID(N'{databaseOwner}[{objectQualifier}CampusEvents_GetEvent]'))
    DROP PROCEDURE {databaseOwner}[{objectQualifier}CampusEvents_GetEvent]
GO

```

```

CREATE PROCEDURE {databaseOwner}[{objectQualifier}CampusEvents_GetEvent]
    @EventId INT
AS
BEGIN
    SET NOCOUNT ON;

    SELECT
        EventId,
        ModuleId,
        Title,
        Description,
        Location,
        EventDate,
        CreatedByUserId,
        CreatedOnDate,
        LastModifiedByUserId,
        LastModifiedOnDate
    FROM {databaseOwner}[{objectQualifier}CampusEvents]
    WHERE EventId = @EventId
END
GO

```

INSERT – Új esemény rögzítése

```
IF EXISTS (SELECT * FROM sysobjects WHERE id =  
OBJECT_ID(N'{databaseOwner}[{objectQualifier}CampusEvents_AddEvent]'))  
    DROP PROCEDURE {databaseOwner}[{objectQualifier}CampusEvents_AddEvent]  
GO
```

```
CREATE PROCEDURE {databaseOwner}[{objectQualifier}CampusEvents_AddEvent]  
    @ModuleId          INT,  
    @Title              NVARCHAR(200),  
    @Description        NVARCHAR(MAX),  
    @Location           NVARCHAR(300),  
    @EventDate          DATETIME,  
    @CreatedByUserId    INT  
AS  
BEGIN  
    SET NOCOUNT ON;  
  
    INSERT INTO {databaseOwner}[{objectQualifier}CampusEvents]  
        (ModuleId, Title, Description, Location, EventDate, CreatedByUserId,  
CreatedOnDate)  
    VALUES  
        (@ModuleId, @Title, @Description, @Location, @EventDate,  
@CreatedByUserId, GETDATE());  
  
    -- Visszaadjuk az újonnan létrehozott rekord azonosítóját  
    SELECT SCOPE_IDENTITY();  
END  
GO
```

UPDATE – Esemény módosítása

```
IF EXISTS (SELECT * FROM sysobjects WHERE id =  
OBJECT_ID(N'{databaseOwner}[{objectQualifier}CampusEvents_UpdateEvent]'))  
    DROP PROCEDURE {databaseOwner}[{objectQualifier}CampusEvents_UpdateEvent]  
GO
```

```
CREATE PROCEDURE {databaseOwner}[{objectQualifier}CampusEvents_UpdateEvent]  
    @EventId          INT,  
    @Title              NVARCHAR(200),  
    @Description        NVARCHAR(MAX),  
    @Location           NVARCHAR(300),  
    @EventDate          DATETIME,  
    @LastModifiedByUserId INT  
AS  
BEGIN  
    SET NOCOUNT ON;  
  
    UPDATE {databaseOwner}[{objectQualifier}CampusEvents]  
    SET  
        Title = @Title,
```

```

        Description          = @Description,
        Location              = @Location,
        EventDate              = @EventDate,
        LastModifiedByUserId   = @LastModifiedByUserId,
        LastModifiedOnDate     = GETDATE()
    WHERE EventId = @EventId
END
GO

DELETE – Esemény törlése
IF EXISTS (SELECT * FROM sysobjects WHERE id =
OBJECT_ID(N'{databaseOwner}[{objectQualifier}CampusEvents_DeleteEvent]'))
    DROP PROCEDURE {databaseOwner}[{objectQualifier}CampusEvents_DeleteEvent]
GO

CREATE PROCEDURE {databaseOwner}[{objectQualifier}CampusEvents_DeleteEvent]
    @EventId INT
AS
BEGIN
    SET NOCOUNT ON;

    DELETE FROM {databaseOwner}[{objectQualifier}CampusEvents]
    WHERE EventId = @EventId
END
GO

```

Ezek a stored procedure-ök az adatbázis belépési pontjai. Minden adatbázis-művelet ide fog befutni, és innentől az SQL Server kezeli a dolgot.

Az Uninstall script

A DNN elvárja, hogy a modul eltávolítása után ne maradjanak felesleges adatbázis-objektumok. Ehhez egy Uninstall.SqlDataProvider fájlt kell készítenünk:

```

-- Tárolt eljárások törlése
DROP PROCEDURE IF EXISTS
{databaseOwner}[{objectQualifier}CampusEvents_GetEvents]
DROP PROCEDURE IF EXISTS
{databaseOwner}[{objectQualifier}CampusEvents_GetEvent]
DROP PROCEDURE IF EXISTS
{databaseOwner}[{objectQualifier}CampusEvents_AddEvent]
DROP PROCEDURE IF EXISTS
{databaseOwner}[{objectQualifier}CampusEvents_UpdateEvent]
DROP PROCEDURE IF EXISTS
{databaseOwner}[{objectQualifier}CampusEvents_DeleteEvent]
GO

-- Tábla törlése
DROP TABLE IF EXISTS {databaseOwner}[{objectQualifier}CampusEvents]
GO

```

6. A Model réteg

A Model réteg az adataink C# reprezentációja. Minél egyszerűbb, annál jobb. A Model nem tud semmiről, nem hivatkozik adatbázisra, nem hivatkozik DNN-re, csak adatot tartalmaz.

Fájl: Models/EventInfo.cs

```
using System;

namespace Corvinus.Modules.CampusEvents.Models
{
    /// <summary>
    /// Egyetlen Campus eseményt reprezentál.
    /// </summary>
    public class EventInfo
    {
        public int EventId { get; set; }
        public int ModuleId { get; set; }
        public string Title { get; set; }
        public string Description { get; set; }
        public string Location { get; set; }
        public DateTime EventDate { get; set; }
        public int CreatedByUserId { get; set; }
        public DateTime CreatedOnDate { get; set; }
        public int? LastModifiedByUserId { get; set; }
        public DateTime? LastModifiedOnDate { get; set; }
    }
}
```

Ez az osztály szándékosan egyszerű. Nincsenek benne metódusok, nincsenek adatbázis-hívások, nincs üzleti logika. Ez egy POCO (Plain Old CLR Object), vagyis nem más, mint adattároló.

Ha szükséges, az adatvalidáció érdekében felvehetünk DataAnnotation attribútumokat is – ezeket a View oldali validációhoz a View Model-ekbe érdemes tenni:

Fájl: Models/EventViewModel.cs

```
using System;
using System.ComponentModel.DataAnnotations;

namespace Corvinus.Modules.CampusEvents.Models
{
    /// <summary>
    /// Az eseménylétrehozó és -szerkesztő form adatait tartalmazza.
    /// Ez egy View Model – kizárólag a View-val való kommunikációra szolgál.
    /// </summary>
    public class EventViewModel
    {

```

```

    public int EventId { get; set; }

    [Required(ErrorMessage = "Az esemény neve kötelező.")]
    [StringLength(200, ErrorMessage = "A cím legfeljebb 200 karakter lehet.")]
    public string Title { get; set; }

    [StringLength(5000)]
    public string Description { get; set; }

    [StringLength(300)]
    public string Location { get; set; }

    [Required(ErrorMessage = "Az esemény dátuma kötelező.")]
    [DataType(DataType.DateTime)]
    public DateTime EventDate { get; set; }
}

```

Figyeljük meg a különbséget az EventInfo az adatbázisból érkező nyers adat, az EventViewModel pedig az, amit a form kezel. Ez a szétválasztás azért fontos, mert a felhasználói felületnek sokszor más adatokra van szüksége, mint amit az adatbázis visszaad.

7. A DataProvider réteg

A DNN hagyományos adatelérési mintája az SqlDataProvider osztályon alapul. Ez a réteg felelős azért, hogy a C# kódból eljussunk az adatbázis stored procedure-jeihez.

Az interfész

Először definiáljunk egy interfészt, hogy a felsőbb rétegek ne fűggenek közvetlenül az SQL implementációtól:

Fájl: Data/IEventDataRepository.cs

```

using System.Collections.Generic;
using Corvinus.Modules.CampusEvents.Models;

namespace Corvinus.Modules.CampusEvents.Data
{
    public interface IEventDataRepository
    {
        IEnumerable<EventInfo> GetEvents(int moduleId);
        EventInfo GetEvent(int eventId);
        int AddEvent(EventInfo eventInfo);
        void UpdateEvent(EventInfo eventInfo);
        void DeleteEvent(int eventId);
    }
}

```

```
}  
}
```

Az SqlDataProvider implementáció

Ez az osztály tartja a tényleges kapcsolatot az adatbázissal. Az IDataReader-en keresztül olvassuk ki az adatokat.

Fájl: Data/SqlDataProvider.cs

```
using System;  
using System.Collections.Generic;  
using System.Data;  
using DotNetNuke.Common.Utilities;  
using DotNetNuke.Framework.Providers;  
using Microsoft.ApplicationBlocks.Data;  
using Corvinus.Modules.CampusEvents.Models;  
  
namespace Corvinus.Modules.CampusEvents.Data  
{  
    public class SqlDataProvider : IEventDataRepository  
    {  
        // A DNN konfiguráció alapján összeállítjuk a connection string-et  
        // és a tábla/SP prefix-et.  
        private const string ProviderType = "data";  
  
        private readonly string _connectionString;  
        private readonly string _databaseOwner;  
        private readonly string _objectQualifier;  
  
        public SqlDataProvider()  
        {  
            // A DNN ProviderConfiguration-ből olvassuk ki a beállításokat  
            var providerConfig =  
ProviderConfiguration.GetProviderConfiguration(ProviderType);  
            var objProvider =  
(Provider)providerConfig.Providers[providerConfig.DefaultProvider];  
  
            _connectionString = Config.GetConnectionString();  
            _databaseOwner = objProvider.Attributes["databaseOwner"];  
            _objectQualifier = objProvider.Attributes["objectQualifier"];  
        }  
  
        // Segédmetódus: összerakja a teljes SP nevet  
        private string GetFullyQualifiedName(string name)  
        {  
            return _databaseOwner + _objectQualifier + "CampusEvents_" +  
name;  
        }  
    }  
}
```

```

// =====
// LEKÉRDEZÉSEK
// =====

public IEnumerable<EventInfo> GetEvents(int moduleId)
{
    var events = new List<EventInfo>();

    // SqlHelper.ExecuteReader: végrehajtja az SP-t és visszaad egy
IDataReader-t
    using (IDataReader reader = SqlHelper.ExecuteReader(
        _connectionString,
        CommandType.StoredProcedure,
        GetFullyQualifiedName("GetEvents"),
        new System.Data.SqlClient.SqlParameter("@ModuleId",
moduleId)))
    {
        while (reader.Read())
        {
            events.Add(FillEventInfo(reader));
        }
    }

    return events;
}

public EventInfo GetEvent(int eventId)
{
    EventInfo eventInfo = null;

    using (IDataReader reader = SqlHelper.ExecuteReader(
        _connectionString,
        CommandType.StoredProcedure,
        GetFullyQualifiedName("GetEvent"),
        new System.Data.SqlClient.SqlParameter("@EventId", eventId)))
    {
        if (reader.Read())
        {
            eventInfo = FillEventInfo(reader);
        }
    }

    return eventInfo;
}

// =====
// ADATMÓDOSÍTÁSOK
// =====

```

```

    public int AddEvent(EventInfo eventInfo)
    {
        // ExecuteScalar-t használunk, mert az SP visszaadja az új
        // azonosítót
        object result = SqlHelper.ExecuteScalar(
            _connectionString,
            CommandType.StoredProcedure,
            GetFullyQualifiedName("AddEvent"),
            new System.Data.SqlClient.SqlParameter("@ModuleId",
eventInfo.ModuleId),
            new System.Data.SqlClient.SqlParameter("@Title",
eventInfo.Title),
            new System.Data.SqlClient.SqlParameter("@Description",
(object)eventInfo.Description ?? DBNull.Value),
            new System.Data.SqlClient.SqlParameter("@Location",
(object)eventInfo.Location ?? DBNull.Value),
            new System.Data.SqlClient.SqlParameter("@EventDate",
eventInfo.EventDate),
            new System.Data.SqlClient.SqlParameter("@CreatedByUserId",
eventInfo.CreatedByUserId));

        return Convert.ToInt32(result);
    }

    public void UpdateEvent(EventInfo eventInfo)
    {
        SqlHelper.ExecuteNonQuery(
            _connectionString,
            CommandType.StoredProcedure,
            GetFullyQualifiedName("UpdateEvent"),
            new System.Data.SqlClient.SqlParameter("@EventId",
eventInfo.EventId),
            new System.Data.SqlClient.SqlParameter("@Title",
eventInfo.Title),
            new System.Data.SqlClient.SqlParameter("@Description",
(object)eventInfo.Description ?? DBNull.Value),
            new System.Data.SqlClient.SqlParameter("@Location",
(object)eventInfo.Location ?? DBNull.Value),
            new System.Data.SqlClient.SqlParameter("@EventDate",
eventInfo.EventDate),
            new
System.Data.SqlClient.SqlParameter("@LastModifiedByUserId",
eventInfo.LastModifiedByUserId));
    }

    public void DeleteEvent(int eventId)
    {
        SqlHelper.ExecuteNonQuery(
            _connectionString,
            CommandType.StoredProcedure,

```

```

        GetFullyQualifiedName("DeleteEvent"),
        new System.Data.SqlClient.SqlParameter("@EventId", eventId));
    }

    // =====
    // SEGÉDMETÓDUSOK
    // =====

    /// <summary>
    /// Az IDataReader aktuális sorából feltölti az EventInfo objektumot.
    /// Ez a metódus felelős az adatbázis oszlopok és a C# tulajdonságok
    összekapcsolásáért.
    /// </summary>
    private static EventInfo FillEventInfo(IDataReader reader)
    {
        return new EventInfo
        {
            EventId      = Null.SetNullInteger(reader["EventId"]),
            ModuleId     = Null.SetNullInteger(reader["ModuleId"]),
            Title        = Null.SetNullString(reader["Title"]),
            Description   = Null.SetNullString(reader["Description"]),
            Location      = Null.SetNullString(reader["Location"]),
            EventDate     = Null.SetNullDateTime(reader["EventDate"]),
            CreatedByUserId =
Null.SetNullInteger(reader["CreatedByUserId"]),
            CreatedOnDate =
Null.SetNullDateTime(reader["CreatedOnDate"]),
            LastModifiedByUserId = reader["LastModifiedByUserId"] ==
DBNull.Value
                                ? (int?)null
                                :
Convert.ToInt32(reader["LastModifiedByUserId"]),
            LastModifiedOnDate = reader["LastModifiedOnDate"] ==
DBNull.Value
                                ? (DateTime?)null
                                :
Convert.ToDateTime(reader["LastModifiedOnDate"])
        };
    }
}

```

Az IDataReader részletezése

Az IDataReader az ADO.NET egyik alapköve. Lényege, hogy az adatbázisból soronként, előre haladva olvas adatot. Ez memóriahatékony, mert nem tölti be az összes adatot egyszerre. A while (reader.Read()) ciklus léptet előre, és minden lépésben az aktuális sort olvashatjuk.

Néhány fontos szempont: - Az `IDataReader`-t mindig `using` blokkban kell használni, hogy a kapcsolat biztosan lezáruljon - Az oszlopok indexe vagy neve alapján érhetők el: `reader["Title"]` vagy `reader[0]` - A `DBNull.Value` kezelése kritikus, ha egy nullable mező null értéket tartalmaz az adatbázisban, a `DNN Null.SetNull*` segédmetódusai kezelik ezt

8. A Service réteg

A Service réteg az a hely, ahol az üzleti szabályok élnek. Ha például egy eseményt csak akkor lehet törölni, ha már elmúlt, ez a logika ide kerül, nem a Controllerbe és nem az adatelérési rétegbe.

Fájl: Services/EventService.cs

```
using System;
using System.Collections.Generic;
using System.Linq;
using Corvinus.Modules.CampusEvents.Data;
using Corvinus.Modules.CampusEvents.Models;

namespace Corvinus.Modules.CampusEvents.Services
{
    public class EventService
    {
        private readonly IEventDataRepository _repository;

        // Dependency Injection: a repository-t kívülről kapjuk,
        // így könnyen cserélhető teszteléskor
        public EventService(IEventDataRepository repository)
        {
            _repository = repository;
        }

        // Kényelmi konstruktor, hogy ne kelljen mindig példányosítani a
        // repository-t
        public EventService() : this(new SqlDataProvider())
        {
        }

        /// <summary>
        /// Az összes jövőbeli eseményt visszaadja dátum szerint rendezve.
        /// </summary>
        public IEnumerable<EventInfo> GetUpcomingEvents(int moduleId)
        {
            return _repository.GetEvents(moduleId)
                .Where(e => e.EventDate >= DateTime.Today)
                .OrderBy(e => e.EventDate);
        }
    }
}
```

```

/// <summary>
/// Összes esemény - adminisztrációs nézetekhez.
/// </summary>
public IEnumerable<EventInfo> GetAllEvents(int moduleId)
{
    return _repository.GetEvents(moduleId)
        .OrderBy(e => e.EventDate);
}

/// <summary>
/// Egyetlen esemény lekérdezése. Ellenőrzi, hogy létezik-e.
/// </summary>
public EventInfo GetEvent(int eventId)
{
    var eventInfo = _repository.GetEvent(eventId);
    if (eventInfo == null)
        throw new ArgumentException($"Nem létezik esemény {eventId}
azonosítóval.");
    return eventInfo;
}

/// <summary>
/// Új esemény mentése - üzleti validációval.
/// </summary>
public int CreateEvent(EventInfo eventInfo)
{
    if (eventInfo == null)
        throw new ArgumentNullException(nameof(eventInfo));

    if (string.IsNullOrEmpty(eventInfo.Title))
        throw new ArgumentException("Az esemény neve nem lehet
üres.");

    if (eventInfo.EventDate < DateTime.Today)
        throw new ArgumentException("Az esemény dátuma nem lehet a
múltban.");

    return _repository.AddEvent(eventInfo);
}

/// <summary>
/// Esemény frissítése.
/// </summary>
public void UpdateEvent(EventInfo eventInfo)
{
    if (eventInfo == null)
        throw new ArgumentNullException(nameof(eventInfo));

    // Ellenőrizzük, hogy a módosítandó rekord valóban létezik

```



```

        var existing = _repository.GetEvent(eventInfo.EventId);
        if (existing == null)
            throw new ArgumentException("A módosítandó esemény nem
található.");

        _repository.UpdateEvent(eventInfo);
    }

    /// <summary>
    /// Esemény törlése - csak jövőbeli esemény törölhető.
    /// </summary>
    public void DeleteEvent(int eventId)
    {
        var eventInfo = _repository.GetEvent(eventId);
        if (eventInfo == null)
            throw new ArgumentException("A törölni kívánt esemény nem
található.");

        _repository.DeleteEvent(eventId);
    }
}

```

A Service réteg szépsége az, hogy teljesen el van különítve az adatbázistól és a web keretrendszertől. Ezt az osztályt ugyanolyan jól lehet unit tesztelni egy konzolalkalmazásban, mint egy webalkalmazásban, épp ezért érdemes ide tenni az üzleti logikát.

9. A Controller

A Controller a legizgalmasabb rész, ő köti össze a többi réteget. Fogadja a HTTP-kéréseket, meghívja a Service-t, és visszaadja a megfelelő View-t.

Fájl: Controllers/EventController.cs

```

using System;
using System.Web.Mvc;
using DotNetNuke.Security;
using DotNetNuke.Web.Mvc.Framework.ActionFilters;
using DotNetNuke.Web.Mvc.Framework.Controllers;
using Corvinus.Modules.CampusEvents.Models;
using Corvinus.Modules.CampusEvents.Services;

namespace Corvinus.Modules.CampusEvents.Controllers
{
    // Ez az attribútum biztosítja, hogy a DNN keret megfelelően
    inicializálódjon
    [DnnHandleError]

```

```

public class EventController : DnnController
{
    private readonly EventService _eventService;

    public EventController()
    {
        _eventService = new EventService();
    }

    // =====
    // LISTÁZÁS
    // =====

    /// <summary>
    /// Főoldal: a jövőbeli események listája.
    /// A modul letöltésekor ez az action fut le először.
    /// </summary>
    [HttpGet]
    public ActionResult Index()
    {
        // A ModuleContext.ModuleId a DNN által biztosított modul-
        // azonosító
        var events =
            _eventService.GetUpcomingEvents(ModuleContext.ModuleId);
        return View(events);
    }

    // =====
    // RÉSZLETEK
    // =====

    [HttpGet]
    public ActionResult Details(int id)
    {
        try
        {
            var eventInfo = _eventService.GetEvent(id);
            return View(eventInfo);
        }
        catch (ArgumentException)
        {
            // Ha nem találjuk az eseményt, visszairányítjuk a listára
            return RedirectToDefaultRoute();
        }
    }

    // =====
    // LÉTREHOZÁS
    // =====

```

```

[HttpGet]
// DnnModuleAuthorize: csak a modul szerkesztési jogával rendelkezők
láthatják
[DnnModuleAuthorize(AccessLevel = SecurityAccessLevel.Edit)]
public ActionResult Edit()
{
    return View(new EventViewModel { EventDate =
DateTime.Today.AddDays(1) });
}

[HttpPost]
[DnnModuleAuthorize(AccessLevel = SecurityAccessLevel.Edit)]
[ValidateAntiForgeryToken]
public ActionResult Edit(EventViewModel viewModel)
{
    if (!ModelState.IsValid)
    {
        // Ha a validáció nem sikerült, visszaküldjük a formot a
hibákkal
        return View(viewModel);
    }

    try
    {
        var eventInfo = new EventInfo
        {
            ModuleId      = ModuleContext.ModuleId,
            Title          = viewModel.Title,
            Description    = viewModel.Description,
            Location        = viewModel.Location,
            EventDate      = viewModel.EventDate,
            CreatedByUserId = User.UserID
        };

        if (viewModel.EventId == 0)
        {
            // Új esemény létrehozása
            _eventService.CreateEvent(eventInfo);
        }
        else
        {
            // Meglévő esemény frissítése
            eventInfo.EventId = viewModel.EventId;
            eventInfo.LastModifiedByUserId = User.UserID;
            _eventService.UpdateEvent(eventInfo);
        }

        // Sikeres mentés után vissza a listára
        return RedirectToDefaultRoute();
    }
}

```

```

    }
    catch (ArgumentException ex)
    {
        ModelState.AddModelError(string.Empty, ex.Message);
        return View(viewModel);
    }
}

// -----
// SZERKESZTÉS (meglévő esemény)
// -----

```

```

[HttpGet]
[DnnModuleAuthorize(AccessLevel = SecurityAccessLevel.Edit)]
public ActionResult EditExisting(int id)
{
    try
    {
        var eventInfo = _eventService.GetEvent(id);

        var viewModel = new EventViewModel
        {
            EventId      = eventInfo.EventId,
            Title        = eventInfo.Title,
            Description   = eventInfo.Description,
            Location      = eventInfo.Location,
            EventDate     = eventInfo.EventDate
        };

        return View("Edit", viewModel);
    }
    catch (ArgumentException)
    {
        return RedirectToDefaultRoute();
    }
}

```

```

// -----
// TÖRLÉS
// -----

```

```

[HttpGet]
[DnnModuleAuthorize(AccessLevel = SecurityAccessLevel.Edit)]
public ActionResult Delete(int id)
{
    try
    {
        var eventInfo = _eventService.GetEvent(id);
        return View(eventInfo);
    }
}

```

```

    }
    catch (ArgumentException)
    {
        return RedirectToDefaultRoute();
    }
}

[HttpPost]
[DnnModuleAuthorize(AccessLevel = SecurityAccessLevel.Edit)]
[ValidateAntiForgeryToken]
[ActionName("Delete")]
public ActionResult DeleteConfirmed(int id)
{
    try
    {
        _eventService.DeleteEvent(id);
    }
    catch (ArgumentException ex)
    {
        // Naplózzuk a hibát, de ne törjük össze az oldalt
        DotNetNuke.Services.Exceptions.Exceptions.LogException(ex);
    }

    return RedirectToDefaultRoute();
}
}
}

```

Amit érdemes megfigyelni a Controllerben

Az `[DnnModuleAuthorize]` attribútum a DNN saját engedélyezési mechanizmusa. A `SecurityAccessLevel.Edit` azt jelenti, hogy az adott action csak azok számára érhető el, akik szerkesztési joggal rendelkeznek az adott modulhoz. Ha valaki nem jogosult, a DNN automatikusan visszairányítja a bejelentkezési oldalra.

A `User.UserID` nem a szokásos ASP.NET User-objektum, a `DnnController` egy `DotNetNuke.Entities.Users.UserInfo` típusú User-t biztosít, ami gazdagabb az egyszerű identity-nél.

10. A View-k

A View-k Razor templates-ek, amelyek ugyanúgy néznek ki, mint egy normál ASP.NET MVC alkalmazásban. A DNN-specifikus rész csak annyi, hogy hozzáférünk a `Dnn` segédobjektumhoz, amely a portál, az oldal és a modul adatait tartalmazza.

Index View – eseménylista

Fájl: Views/Event/Index.cshtml

```

@using DotNetNuke.Web.Mvc.Helpers
@model IEnumerable<Corvinus.Modules.CampusEvents.Models.EventInfo>

@{
    // A nézet nevét a DNN a modul fejlécébe írja
    Layout = null;
}

<div class="campus-events-module">
    <div class="events-header">
        <h2>Közelgő események</h2>

        @* Szerkesztési gombok csak szerkesztési módban jelennek meg *@
        @if (Dnn.ModuleContext.IsEditable)
        {
            <a href="@Url.Action("Edit", "Event")" class="btn btn-primary
btn-sm">
                + Új esemény
            </a>
        }
    </div>

    @if (!Model.Any())
    {
        <div class="alert alert-info">
            Jelenleg nincsenek közelgő események.
        </div>
    }
    else
    {
        <div class="events-list">
            @foreach (var evt in Model)
            {
                <div class="event-card">
                    <div class="event-date">
                        <span class="day">@evt.EventDate.Day</span>
                        <span
class="month">@evt.EventDate.ToString("MMM")</span>
                    </div>
                    <div class="event-details">
                        <h3>
                            <a href="@Url.Action("Details", "Event", new { id
= evt.EventId })">
                                @evt.Title
                            </a>
                        </h3>
                        @if (!string.IsNullOrEmpty(evt.Location))
                        {
                            <p class="location">
                                <i class="fa fa-map-marker"></i>

```

@evt.Location

```
        </p>
    }
</div>
@if (Dnn.ModuleContext.IsEditable)
{
    <div class="event-actions">
        <a href="@Url.Action("EditExisting", "Event", new
{ id = evt.EventId })"
        class="btn btn-xs btn-default">Szerkesztés</a>
        <a href="@Url.Action("Delete", "Event", new { id
= evt.EventId })"
        class="btn btn-xs btn-danger">Törlés</a>
    </div>
}
</div>
}
</div>
}
```

Esemény részletei

Fájl: Views/Event/Details.cshtml

@model Corvinus.Modules.CampusEvents.Models.EventInfo

```
<div class="campus-events-module">
    <div class="event-detail">
        <h2>@Model.Title</h2>

        <dl class="dl-horizontal">
            <dt>Dátum</dt>
            <dd>@Model.EventDate.ToString("yyyy. MMMM dd. HH:mm")</dd>

            @if (!string.IsNullOrEmpty(Model.Location))
            {
                <dt>Helyszín</dt>
                <dd>@Model.Location</dd>
            }
        </dl>

        @if (!string.IsNullOrEmpty(Model.Description))
        {
            <div class="event-description">
                <p>@Model.Description</p>
            </div>
        }

        <a href="@Url.Action("Index", "Event")" class="btn btn-default">
```

```

        &laquo; Vissza a listára
    </a>
</div>
</div>

```

Esemény létrehozása/szerkesztése

Fájl: Views/Event/Edit.cshtml

```
@using DotNetNuke.Web.Mvc.Helpers
```

```
@model Corvinus.Modules.CampusEvents.Models.EventViewModel
```

```

<div class="campus-events-module">
    <h2>@(Model.EventId == 0 ? "Új esemény" : "Esemény szerkesztése")</h2>

    @using (Html.BeginForm("Edit", "Event", FormMethod.Post, new { @class =
"form-horizontal" }))
    {
        @Html.AntiForgeryToken()

        @* Rejtett mező az EventId-nak szerkesztési módban *@
        @Html.HiddenFor(m => m.EventId)

        <div class="form-group @(Html.HasError("Title") ? "has-error" : "")">
            @Html.LabelFor(m => m.Title, "Esemény neve", new { @class = "col-
sm-3 control-label" })
            <div class="col-sm-9">
                @Html.TextBoxFor(m => m.Title, new { @class = "form-control",
placeholder = "pl. Gólyabál 2025" })
                @Html.ValidationMessageFor(m => m.Title, "", new { @class =
"text-danger" })
            </div>
        </div>

        <div class="form-group @(Html.HasError("EventDate") ? "has-error" :
"")">
            @Html.LabelFor(m => m.EventDate, "Dátum és idő", new { @class =
"col-sm-3 control-label" })
            <div class="col-sm-9">
                @Html.TextBoxFor(m => m.EventDate, "{0:yyyy-MM-ddTHH:mm}",
                new { @class = "form-control", type = "datetime-local" })
                @Html.ValidationMessageFor(m => m.EventDate, "", new { @class
= "text-danger" })
            </div>
        </div>

        <div class="form-group">
            @Html.LabelFor(m => m.Location, "Helyszín", new { @class = "col-
sm-3 control-label" })
            <div class="col-sm-9">

```



```

        @Html.TextBoxFor(m => m.Location, new { @class = "form-control", placeholder = "pl. A épület, 101-es terem" })
    </div>
</div>

<div class="form-group">
    @Html.LabelFor(m => m.Description, "Leírás", new { @class = "col-sm-3 control-label" })
    <div class="col-sm-9">
        @Html.TextAreaFor(m => m.Description, 5, 0, new { @class = "form-control" })
    </div>
</div>

    @* Összefoglaló validációs hibák *@
    @Html.ValidationSummary(true, "Kérjük javítsd az alábbi hibákat:", new { @class = "alert alert-danger" })

    <div class="form-group">
        <div class="col-sm-offset-3 col-sm-9">
            <button type="submit" class="btn btn-primary">Mentés</button>
            <a href="@Url.Action("Index", "Event")" class="btn btn-default">Mégse</a>
        </div>
    </div>
}
</div>

```

Törlés megerősítése

Fájl: Views/Event/Delete.cshtml

```

@model Corvinus.Modules.CampusEvents.Models.EventInfo

<div class="campus-events-module">
    <div class="alert alert-danger">
        <h3>Biztosan törlöd ezt az eseményt?</h3>
        <p><strong>@Model.Title</strong></p>
        <p>@Model.EventDate.ToString("yyyy. MMMM dd.") - @Model.Location</p>
    </div>

    @using (Html.BeginForm("Delete", "Event", FormMethod.Post))
    {
        @Html.AntiForgeryToken()
        @Html.Hidden("id", Model.EventId)

        <button type="submit" class="btn btn-danger">Igen, törlöm</button>
        <a href="@Url.Action("Index", "Event")" class="btn btn-default">Mégse</a>
    }

```

```
}  
</div>
```

11. A Manifest fájl (.dnn)

A .dnn manifest fájl írja le a DNN-nek, hogy milyen névvel, verzióval és összetevőkkel rendelkezik a modulunk. Nélküle a DNN nem tudja telepíteni a modult.

Fájl: CampusEvents.dnn

```
<dotnetnuke type="Package" version="5.0">  
  <packages>  
    <package name="Corvinus_CampusEvents"  
      type="Module"  
      version="01.00.00">  
  
      <friendlyName>Campus Events</friendlyName>  
      <description>Kampusz eseménykezelő modul MVC  
architektúrával</description>  
  
      <owner>  
        <name>Corvinus</name>  
        <organization>Universitas Kft.</organization>  
        <url>https://Corvinus.hu</url>  
        <email>support@Corvinus.hu</email>  
      </owner>  
  
      <license src="License.txt" />  
      <releaseNotes src="ReleaseNotes.txt" />  
  
      <components>  
  
        <!-- 1. A modul regisztrálása a DNN-ben -->  
        <component type="Module">  
          <desktopModule>  
            <moduleName>Corvinus_CampusEvents</moduleName>  
            <foldername>CampusEvents</foldername>  
            <businessControllerClass />  
            <supportedFeatures />  
  
            <moduleDefinitions>  
              <moduleDefinition>  
                <friendlyName>Campus Events</friendlyName>  
                <defaultCacheTime>0</defaultCacheTime>  
  
                <moduleControls>  
                  <!-- Az MVC modul fő belépési pontja -->  
                  <moduleControl>
```

```

        <controlKey />

<controlSrc>mvc/Corvinus_CampusEvents/Event/Index</controlSrc>

<supportsPartialRendering>False</supportsPartialRendering>
    <controlTitle>Campus Events</controlTitle>
    <controlType>View</controlType>
    <iconFile />
    <helpUrl />
    <viewOrder>0</viewOrder>
</moduleControl>
</moduleControls>

</moduleDefinition>
</moduleDefinitions>
</desktopModule>
</component>

<!-- 2. Az assembly (Lefordított DLL) -->
<component type="Assembly">
    <assemblies>
        <basePath>bin</basePath>
        <assembly>
            <path>bin</path>
            <name>Corvinus.CampusEvents.dll</name>
            <version>01.00.00</version>
        </assembly>
    </assemblies>
</component>

<!-- 3. A fájlok (View-k, CSS, stb.) -->
<component type="File">
    <files>
        <basePath>DesktopModules/MVC/CampusEvents</basePath>
        <sourceFiles>
            <sourceFile>
                <name>Views\Event\Index.cshtml</name>
            </sourceFile>
            <sourceFile>
                <name>Views\Event\Details.cshtml</name>
            </sourceFile>
            <sourceFile>
                <name>Views\Event\Edit.cshtml</name>
            </sourceFile>
            <sourceFile>
                <name>Views\Event\Delete.cshtml</name>
            </sourceFile>
        </sourceFiles>
    </files>
</component>

```

```

    <!-- 4. Az adatbázis scriptek -->
    <component type="Script">
        <scripts>

<basePath>DesktopModules/MVC/CampusEvents/Providers/DataProviders/SqlDataProv
ider</basePath>
            <script type="Install">
                <path>.</path>
                <name>01.00.00.SqlDataProvider</name>
                <version>01.00.00</version>
            </script>
            <script type="UnInstall">
                <path>.</path>
                <name>Uninstall.SqlDataProvider</name>
            </script>
        </scripts>
    </component>

</components>
</package>
</packages>
</dotnetnuke>

```

A manifest legfontosabb része a <controlSrc> sor:
mvc/Corvinus_CampusEvents/Event/Index. Ez mondja meg a DNN-nek, hogy amikor betölti a modult, az MVC routing alapján az EventController Index action-jét kell meghívni.

12. Build és Deploy

A build folyamat

A DNN MVC modul egy sima .NET osztálykönyvtár (Class Library). A build során a Visual Studio egy .dll fájlt hoz létre. Ezt a DLL-t kell a DNN-nek átadni, együtt a View fájlokkal és az SQL scriptekkel.

A telepítési csomag egy .zip fájl, amelynek belső struktúrája pontosan megfelel annak, amit a manifest fájlban leírtunk.

Post-build esemény Visual Studióban

A csomagolást automatizálhatjuk a Visual Studio **Post-build Event** funkciójával. A projekt Properties → Build Events menüpontban adjuk meg:

```

cd $(ProjectDir)
"C:\Program Files\7-Zip\7z.exe" a -tzip
"$(ProjectDir)\Install\CampusEvents_01.00.00_Install.zip" ^

```

```
"$(TargetPath)" ^  
"Views\" ^  
"Providers\" ^  
"CampusEvents.dnn"
```

Telepítés DNN-be

1. Nyissuk meg a DNN adminisztrátori felületet: **Host → Extensions**
2. Kattintsunk az **Install Extension Wizard** gombra
3. Töltsük fel a .zip csomagot
4. A DNN végigfuttatja a telepítési folyamatot:
 - Bemásolja az assembly-t a bin mappába
 - Bemásolja a View fájlokat
 - Lefuttatja az SQL scripteket
 - Regisztrálja a modult az adatbázisban
5. Sikeres telepítés után az **Edit Page** módban hozzáadhatjuk az oldalhoz a modult: **Add Module → Campus Events**

Verziófrissítések

Ha a modul egy újabb verzióját adjuk ki (pl. 01.01.00), annyi a teendő, hogy: - Létrehozunk egy 01.01.00.SqlDataProvider fájlt a változásokkal (pl. ALTER TABLE) - Frissítjük a manifest-ben a verziószámot - A DNN automatikusan csak az előző verzió óta szükséges scripteket futtatja

13. Tipikus hibák és tanulságok

Fejlesztés közben mindenki belefut bizonyos tipikus hibákba. Az alábbiakban összegyűjtöttem a leggyakoribbakat, hogy elkerülhetők legyenek.

1. Fat Controller

Ez az egyik leggyakoribb hiba, főleg az új fejlesztők körében. A Controller elkezd egyre több logikát felvenni, amíg végül minden benne van, az adatbázis-hívástól az üzleti szabályokig.

Rossz:

```
public ActionResult Edit(EventViewModel viewModel)  
{  
    // Controller-be írt adatbázis-hívás - TILOS!  
    string connStr = Config.GetConnectionString();  
    using (var conn = new SqlConnection(connStr))  
    {  
        conn.Open();  
        var cmd = new SqlCommand("INSERT INTO Events...", conn);  
        cmd.ExecuteNonQuery();  
    }  
}
```

```
        return RedirectToDefaultRoute();  
    }
```

Helyes: Az adatbázis-hívás a DataProvider-ben van, az üzleti logika a Service-ben. A Controller csak koordinál.

2. DBNull nem kezelése

Ha egy nullable mező null értékű az adatbázisban, és ezt nem kezeljük az IDataReader-ben, InvalidCastException-t kapunk futásidőben.

Rossz:

```
Description = (string)reader["Description"] // Crash, ha Description NULL az  
adatbázisban
```

Helyes:

```
Description = reader["Description"] == DBNull.Value ? null :  
(string)reader["Description"]  
// vagy DNN helper segítségével:  
Description = Null.SetNullString(reader["Description"])
```

3. ModuleId hiánya lekérdezéseknél

Egy DNN oldalon ugyanabból a modulból akár több példány is lehet. Ha nem szűrünk ModuleId-ra, az összes modul az összes eseményt mutatja, nem azt, ami hozzá tartozik.

Rossz:

```
SELECT * FROM CampusEvents -- Minden rekord, minden modulból!
```

Helyes:

```
SELECT * FROM CampusEvents WHERE ModuleId = @ModuleId
```

4. Verziókövetés hiánya az SQL scriptekben

Ha az adatbázist közvetlenül módosítjuk (nem script révén), az más fejlesztői vagy teszt-éles környezetben nem fog érvényesülni. Minden séma-módosítás egy új .SqlDataProvider fájlba kerüljön, a megfelelő verziószámmal.

5. Körkörös függőség a rétegek között

Ha a Service réteg importálja a Controller névteret, vagy a DataProvider importálja a Service névteret, az komoly architektúrális hiba. A függőségek mindig felülről lefelé mutatnak:

Controller → Service → DataProvider → SQL

15. API használat DNN MVC modulban

A DNN modulok nemcsak HTML oldalakat tudnak kiszolgálni, hanem JSON API végpontokat is, ez teszi lehetővé, hogy a modult AJAX hívásokkal, egyoldalas alkalmazásokból vagy külső rendszerekből is elérjük.

DNN Web API, a DnnApiController

DNN-ben az API végpontokat nem egy sima MVC Controller, hanem a DnnApiController valósítja meg, amely a System.Web.Http.ApiController-re épül. A DNN ezt a saját jogosultság-kezelőjével és kontextusával egészíti ki.

Fájl: Controllers/EventApiController.cs

```
using System;
using System.Collections.Generic;
using System.Net;
using System.Net.Http;
using System.Web.Http;
using DotNetNuke.Security;
using DotNetNuke.Web.Api;
using Corvinus.Modules.CampusEvents.Models;
using Corvinus.Modules.CampusEvents.Services;

namespace Corvinus.Modules.CampusEvents.Controllers
{
    // A DNN routing megtalálja ezt a controllert:
    // /API/CampusEvents/EventApi/{action}
    public class EventApiController : DnnApiController
    {
        private readonly EventService _eventService;

        public EventApiController()
        {
            _eventService = new EventService();
        }

        // -----
        // GET /API/CampusEvents/EventApi/GetEvents?moduleId=42
        // -----
    }
}
```

```

[HttpGet]
[AllowAnonymous] // Publikus eseménylista - nem kell bejelentkezés
public HttpResponseMessage GetEvents(int moduleId)
{
    try
    {
        var events = _eventService.GetUpcomingEvents(moduleId);
        return Request.CreateResponse(HttpStatusCode.OK, events);
    }
    catch (Exception ex)
    {
        DotNetNuke.Services.Exceptions.Exceptions.LogException(ex);
        return
Request.CreateErrorResponse(HttpStatusCode.InternalServerError, "Hiba az
események lekérdezésekor.");
    }
}

// -----
// GET /API/CampusEvents/EventApi/GetEvent?eventId=7
// -----

[HttpGet]
[AllowAnonymous]
public HttpResponseMessage GetEvent(int eventId)
{
    try
    {
        var eventInfo = _eventService.GetEvent(eventId);
        return Request.CreateResponse(HttpStatusCode.OK, eventInfo);
    }
    catch (ArgumentException)
    {
        return Request.CreateErrorResponse(HttpStatusCode.NotFound,
"Az esemény nem található.");
    }
}

// -----
// POST /API/CampusEvents/EventApi/CreateEvent
// Body: { "title": "...", "eventDate": "...", ... }
// -----

[HttpPost]
[ValidateAntiForgeryToken]
[DnnModuleAuthorize(AccessLevel = SecurityAccessLevel.Edit)]
public HttpResponseMessage CreateEvent([FromBody] EventViewModel
viewModel)
{

```



```

        if (!ModelState.IsValid)
        {
            return Request.CreateErrorResponse(HttpStatusCode.BadRequest,
ModelState);
        }

        try
        {
            var eventInfo = new EventInfo
            {
                ModuleId      = ActiveModule.ModuleID,
                Title          = viewModel.Title,
                Description     = viewModel.Description,
                Location        = viewModel.Location,
                EventDate       = viewModel.EventDate,
                CreatedByUserId = UserInfo.UserID
            };

            int newId = _eventService.CreateEvent(eventInfo);

            // 201 Created + az új rekord URL-je
            return Request.CreateResponse(HttpStatusCode.Created, new {
EventId = newId });
        }
        catch (ArgumentException ex)
        {
            return Request.CreateErrorResponse(HttpStatusCode.BadRequest,
ex.Message);
        }
    }

    // -----
    // DELETE /API/CampusEvents/EventApi/DeleteEvent?eventId=7
    // -----

    [HttpDelete]
    [ValidateAntiForgeryToken]
    [DnnModuleAuthorize(AccessLevel = SecurityAccessLevel.Edit)]
    public HttpResponseMessage DeleteEvent(int eventId)
    {
        try
        {
            _eventService.DeleteEvent(eventId);
            return Request.CreateResponse(HttpStatusCode.OK, new {
message = "Esemény törölve." });
        }
        catch (ArgumentException ex)
        {
            return Request.CreateErrorResponse(HttpStatusCode.NotFound,
ex.Message);
        }
    }

```

```

    }
  }
}

```

RouteMapper

A DNN nem találja meg automatikusan a Web API controllereket. Manuálisan be kell jegyeznünk a route-okat. Ehhez egy RouteMapper osztály szükséges:

Fájl: RouteMapper.cs

```

using DotNetNuke.Web.Api;

namespace Corvinus.Modules.CampusEvents
{
    public class RouteMapper : IServiceRouteMapper
    {
        public void RegisterRoutes(IMapRoute mapRouteManager)
        {
            // Az összes EventApiController action elérhetővé válik az alábbi
            // prefix alatt:
            // /API/CampusEvents/EventApi/{action}
            mapRouteManager.MapHttpRoute(
                moduleFolderName: "CampusEvents",
                routeName:       "default",
                url:               "{controller}/{action}",
                defaults:          new { },
                namespaces:        new[] {
"Corvinus.Modules.CampusEvents.Controllers" }
            );
        }
    }
}

```

A DNN automatikusan megkeresi és betölti az IServiceRouteMapper interfészt implementáló osztályokat az assembly-ből, tehát nincs más teendő.

Az API hívása JavaScript oldalról

A View-ban AJAX-szal is lekérdezhethetjük az eseményeket, például egy dinamikus lista megjelenítéséhez:

```

@* Views/Event/Index.cshtml - AJAX-os változat *@
<div id="events-container">
    <p>Töltés...</p>
</div>

<script>
    // A moduleId-t a Razor template-ből vesszük át
    var moduleId = @Dnn.ModuleContext.ModuleId;

```

```

    // DNN API hívás: a headers-ben szerepelnie kell a
    RequestVerificationToken-nek
    // (DNN elvárja, kivéve AllowAnonymous végpontoknál)
    $.get('/API/CampusEvents/EventApi/GetEvents?moduleId=' + moduleId)
        .done(function (data) {
            var html = '';
            data.forEach(function (evt) {
                html += '<div class="event-card">';
                html += '<h3>' + evt.Title + '</h3>';
                html += '<p>' + evt.Location + '</p>';
                html += '</div>';
            });
            $('#events-container').html(html);
        })
        .fail(function () {
            $('#events-container').html('<p class="text-danger">Hiba az
            adatok betöltésekor.</p>');
        });
</script>

```

16. Fájlfeltöltés és fájlkezelés DNN-ben

A fájlkezelés az egyik olyan terület, ahol a DNN saját rendszere lényegesen különbözik egy sima ASP.NET alkalmazástól. A DNN rendelkezik egy beépített fájlkezelő szolgáltatással (IFileManager, IFolderManager), és erősen ajánlott ezt használni a nyers fájlrendszer- vagy adatbázis-műveletek helyett.

Miért a DNN saját fájlkezelője?

A DNN fájlrendszere portálonként szeparál, minden portálhoz saját médiakönyvtár tartozik, és a fájlok meta-adatai (fájlnév, méret, mime-type, tulajdonos) az adatbázisban is nyilván vannak tartva. Ezáltal: - Az adminisztrátor a DNN File Manager felületén is látja és kezelheti a feltöltött fájlokat - A fájlokhoz DNN-szintű jogosultság rendelhető (ki láthatja, ki töltheti le) - A fájlok URL-je konzisztens és portálfüggő

A modell kiegészítése – fájlhivatkozás az eseményen

Bővítsük ki az adatbázis-sémánkat és a modelünket úgy, hogy egy eseményhez csatolmány is tartozhasson (pl. PDF meghívó):

SQL – tábla bővítése:

```

-- 01.01.00.SqlDataProvider - verziófrissítő script
ALTER TABLE {databaseOwner}[{objectQualifier}CampusEvents]
    ADD [AttachmentFileId] INT NULL;
GO

```

Model kiegészítés:

```

public class EventInfo
{
    // ... (korábbi mezők)
    public int? AttachmentFileId { get; set; } // DNN fájlrendszer-azonosító
}

```

A fájlfeltöltő action

A fájlba írási logikát a Service-be tehetjük, hogy a Controller ne tudjon az IFileManager-ról:

Fájl: Services/EventFileService.cs

```

using System.IO;
using System.Web;
using DotNetNuke.Entities.Portals;
using DotNetNuke.Services.FileSystem;

namespace Corvinus.Modules.CampusEvents.Services
{
    public class EventFileService
    {
        /// <summary>
        /// Feltölt egy fájlt a DNN portál "CampusEvents" mappájába,
        /// és visszaadja a DNN fájl-azonosítót.
        /// </summary>
        public int UploadAttachment(HttpPostedFileBase file, int portalId)
        {
            var folderManager = FolderManager.Instance;
            var fileManager = FileManager.Instance;

            // A célmappa: /Portals/0/CampusEvents/ (ha nem létezik,
            // létrehozuk)
            const string folderPath = "CampusEvents/";
            IFolderInfo folder = folderManager.GetFolder(portalId,
            folderPath)
            ?? folderManager.AddFolder(portalId,
            folderPath);

            // Biztonságos fájlnev: szóközők és különleges karakterek
            // eltávolítása
            string safeName = Path.GetFileName(file.FileName)
                .Replace(" ", "_");

            // Fájl mentése a DNN fájlrendszerbe
            IFileInfo savedFile = fileManager.AddFile(
                folder,
                safeName,
                file.InputStream,
                overwrite: true
            );
        }
    }
}

```

```

        return savedFile.FileId;
    }

    /// <summary>
    /// Visszaadja a fájl Letöltési URL-jét a DNN FileId alapján.
    /// </summary>
    public string GetDownloadUrl(int fileId)
    {
        var fileInfo = FileManager.Instance.GetFile(fileId);
        if (fileInfo == null) return null;

        // A DNN saját Letöltési URL-je - kezeli a jogosultságot is
        return FileManager.Instance.GetUrl(fileInfo);
    }

    /// <summary>
    /// Törli a fájlt a DNN fájlrendszerből.
    /// </summary>
    public void DeleteAttachment(int fileId)
    {
        var fileInfo = FileManager.Instance.GetFile(fileId);
        if (fileInfo != null)
        {
            FileManager.Instance.DeleteFile(fileInfo);
        }
    }
}

```

A Controller kiegészítése

Az Edit action-t bővítjük, hogy fogadja a feltöltött fájlt:

Fájl: Controllers/EventController.cs – Edit POST bővítve

```

[HttpPost]
[DnnModuleAuthorize(AccessLevel = SecurityAccessLevel.Edit)]
[ValidateAntiForgeryToken]
public ActionResult Edit(EventViewModel viewModel, HttpPostedFileBase attachment)
{
    if (!ModelState.IsValid)
    {
        return View(viewModel);
    }

    try
    {
        var fileService = new EventFileService();
    }
}

```

```

int? fileId = null;

// Ha érkezett fájl, feltöltjük
if (attachment != null && attachment.ContentLength > 0)
{
    // Csak PDF és képfájlok engedélyezve
    var allowed = new[] { "application/pdf", "image/jpeg",
"image/png" };
    if (!Array.Exists(allowed, t => t == attachment.ContentType))
    {
        ModelState.AddModelError("attachment", "Csak PDF, JPG és PNG
fájl tölthető fel.");
        return View(viewModel);
    }

    // Méretlimit: 10 MB
    if (attachment.ContentLength > 10 * 1024 * 1024)
    {
        ModelState.AddModelError("attachment", "A fájl mérete nem
haladhatja meg a 10 MB-ot.");
        return View(viewModel);
    }

    fileId = fileService.UploadAttachment(attachment,
PortalSettings.PortalId);
}

var eventInfo = new EventInfo
{
    ModuleId      = ModuleContext.ModuleId,
    Title         = viewModel.Title,
    Description   = viewModel.Description,
    Location      = viewModel.Location,
    EventDate     = viewModel.EventDate,
    CreatedByUserId = User.UserID,
    AttachmentFileId = fileId
};

if (viewModel.EventId == 0)
{
    _eventService.CreateEvent(eventInfo);
}
else
{
    eventInfo.EventId = viewModel.EventId;
    eventInfo.LastModifiedByUserId = User.UserID;
    _eventService.UpdateEvent(eventInfo);
}

```

```

        return RedirectToDefaultRoute();
    }
    catch (ArgumentException ex)
    {
        ModelState.AddModelError(string.Empty, ex.Message);
        return View(viewModel);
    }
}

```

A View kiegészítése

A form-nak enctype="multipart/form-data" kell, különben a fájl nem kerül átvitelre:

@* Views/Event/Edit.cshtml – fájlfeltöltő rész hozzáadva *@

```

@using (Html.BeginForm("Edit", "Event", FormMethod.Post,
    new { @class = "form-horizontal", enctype = "multipart/form-data" }))
{
    @Html.AntiForgeryToken()
    @Html.HiddenFor(m => m.EventId)

    @* ... egyéb mezők ... *@

    <div class="form-group">
        <label class="col-sm-3 control-label">Csatolmány (PDF / JPG /
PNG)</label>
        <div class="col-sm-9">
            <input type="file" name="attachment"
accept=".pdf,.jpg,.jpeg,.png" class="form-control" />
            <p class="help-block">Opcionális. Max. 10 MB.</p>
        </div>
    </div>

    <div class="form-group">
        <div class="col-sm-offset-3 col-sm-9">
            <button type="submit" class="btn btn-primary">Mentés</button>
            <a href="@Url.Action("Index", "Event")" class="btn btn-
default">Mégse</a>
        </div>
    </div>
}

```

A feltöltött fájl megjelenítése és letöltése

A Details nézetben megjelenítjük a letöltési linket, ha van csatolmány:

@* Views/Event/Details.cshtml – csatolmány link *@

```

@{
    var fileService = new
Corvinus.Modules.CampusEvents.Services.EventFileService();
}

```

```

@if (Model.AttachmentFileId.HasValue)
{
    string downloadUrl =
fileService.GetDownloadUrl(Model.AttachmentFileId.Value);
    if (!string.IsNullOrEmpty(downloadUrl))
    {
        <div class="event-attachment">
            <a href="@downloadUrl" target="_blank" class="btn btn-default
btn-sm">
                <i class="fa fa-download"></i> Csatolmány letöltése
            </a>
        </div>
    }
}

```

Admin letöltési API végpont

Ha az adminisztrátor egy külön API végponton keresztül is el szeretné érni a csatolmányokat (pl. egy listában, ahol minden esemény mellé letöltési link kell), bővíthetjük az EventApiController-t:

```

// GET /API/CampusEvents/EventApi/GetAttachmentUrl?eventId=7
[HttpGet]
[DnnModuleAuthorize(AccessLevel = SecurityAccessLevel.Edit)]
public HttpResponseMessage GetAttachmentUrl(int eventId)
{
    try
    {
        var eventInfo = _eventService.GetEvent(eventId);

        if (!eventInfo.AttachmentFileId.HasValue)
        {
            return Request.CreateErrorResponse(HttpStatusCode.NotFound,
"Nincs csatolmány ehhez az eseményhez.");
        }

        var fileService = new EventFileService();
        string url =
fileService.GetDownloadUrl(eventInfo.AttachmentFileId.Value);

        return Request.CreateResponse(HttpStatusCode.OK, new { DownloadUrl =
url });
    }
    catch (ArgumentException ex)
    {
        return Request.CreateErrorResponse(HttpStatusCode.NotFound,
ex.Message);
    }
}

```


A fájlok helye a szerveren

A DNN a feltöltött fájlokat a portál média-könyvtárába helyezi:

[DNN gyökér]/Portals/0/CampusEvents/

Az adminisztrátor a DNN **Admin** → **File Manager** felületén ezen a mappán belül látja a feltöltött fájlokat, átnevezheti, törölheti, vagy áthelyezheti őket. A DNN jogosultságrendszere alapján megadható, hogy a mappa nyilvános-e (bárki letöltheti), vagy csak bejelentkezett / csak adminok láthatják.

Dokumentum vége – Verzió: 1.1

Felhasznált technológiák és verziók

DotNetNuke Platform 9.x | ASP.NET MVC 5 | ASP.NET Web API 2 | C# 8.0 | .NET Framework 4.7.2 | SQL Server 2019 | Visual Studio 2022